

课程笔记

# [LLM Agents SP25] Informal+Formal Math Reasoning —Sean Welleck

基于公开课程资料整理

2025



视频作者/频道: Berkeley RDI

发布日期: 2025

视频时长: 约 60 分钟

项目主页: <https://github.com/hqh1025/ai-course-notes>

# 目录

|                                            |          |
|--------------------------------------------|----------|
| <b>1 引言：非形式化与形式化数学推理的桥梁</b>                | <b>2</b> |
| 1.1 形式化证明助手 (Lean) 简介                      | 2        |
| 1.2 形式化数学对 AI 的重要性                         | 2        |
| <b>2 第一部分：将非形式化思考融入形式化证明——LeanSTaR</b>     | <b>2</b> |
| 2.1 动机：超越纯形式化训练                            | 2        |
| 2.2 LeanSTaR 算法                            | 3        |
| 2.3 实验结果                                   | 3        |
| 2.4 本章小结                                   | 3        |
| <b>3 第二部分：非形式化证明——草稿-草图-证明</b>             | <b>4</b> |
| 3.1 动机：结合高级推理与低级证明                         | 4        |
| 3.2 Draft, Sketch, Prove (DSP) 方法          | 4        |
| 3.3 实验结果                                   | 4        |
| <b>4 第三部分：为 Lean 构建 Hammer——LeanHammer</b> | <b>4</b> |
| 4.1 Hammer 管道架构                            | 4        |
| 4.2 实验结果                                   | 5        |
| 4.3 本章小结                                   | 5        |
| <b>5 第四部分：迈向研究级数学的 AI 辅助</b>               | <b>5</b> |
| 5.1 研究级形式化的特殊挑战                            | 5        |
| 5.2 MiniCTX 基准测试                           | 5        |
| 5.3 实用工具：LLMLean                           | 5        |
| <b>6 总结与延伸</b>                             | <b>6</b> |
| 6.1 三条桥接路径                                 | 6        |
| 6.2 拓展阅读                                   | 6        |

# 1 引言：非形式化与形式化数学推理的桥梁

本讲由 CMU 的 Sean Welleck 主讲，探讨如何利用 AI 弥合非形式化数学 (informal math) 与形式化数学 (formal math) 之间的鸿沟，结合两者各自的优势。

## 两种数学表示的对比

**非形式化数学：**以自然语言（文本、LaTeX、图像等）表示，极为灵活，但无法保证正确性。LLM 在中间步骤中可能犯微妙错误。

**形式化数学：**以编程语言（Lean、Coq、Isabelle）编写，代码可编译 = 证明正确。提供绝对的正确性保证，但编写门槛高、过程繁琐。

## 1.1 形式化证明助手 (Lean) 简介

Lean 是一种交互式证明助手 (Interactive Theorem Prover)，核心概念：

- **定理声明：**用形式化语言表述需要证明的命题
- **证明状态 (Proof State)：**当前上下文（假设）和待证目标
- **策略 (Tactic)：**每一步应用的推理规则，改变证明状态
- **QED：**所有目标消解后证明完成

## Mathlib：形式化数学的基础设施

Mathlib 是 Lean 的数学库项目，众多贡献者共同构建数学基础定义和定理。它使形式化证明可以复用已有结果，数学家 Terence Tao 已成功将其论文定理在 Lean 中形式化。

## 1.2 形式化数学对 AI 的重要性

1. **输出可验证：**防止不正确的数学推理
2. **完美奖励信号：**可用于 RL 训练——证明编译通过即正确
3. **推理测试平台：**从简单到任意困难的推理问题

形式化定理证明近年进展迅速：从 GPT-f (2020) 到 DeepSeek-Prover，mini-F2F 基准测试成绩持续飙升。

# 2 第一部分：将非形式化思考融入形式化证明——LeanSTaR

## 2.1 动机：超越纯形式化训练

传统方法仅在形式化证明数据上训练模型（状态  $\rightarrow$  下一步策略），但这可能不足以学到产生代码所需的思维过程。

## LeanSTaR 的核心思想

训练模型在每步形式化证明之前，先生成自由形式的**非形式化思考** (informal thought)：

- 模型可以分解问题、规划下一步、执行非形式化计算
- 思考帮助多样化搜索空间
- 类似 Chain-of-Thought，增强模型的表达能力

## 2.2 LeanSTaR 算法

两阶段训练：

### 阶段一：初始化

1. 对已有证明（如 Mathlib），给定状态和下一步策略，让 LLM 回顾性地生成描述该转换的思考
2. 收集数十万条（状态, 思考, 策略）样本
3. 训练初始模型学会在策略前生成思考

### 阶段二：强化学习 (Expert Iteration)

1. 模型并行生成多个完整证明
2. Lean 检验证明是否正确
3. 收集成功的轨迹加入训练集
4. 在扩展数据集上训练新模型
5. 迭代此过程

## 树搜索在加入思考后的困难

传统的 Best-First Search 在加入非形式化思考后效果不佳——难以对思考进行准确评分。LeanSTaR 改用简单的**并行采样策略**：多条路径并行生成，遇错重试，成功即停。

## 2.3 实验结果

- LeanSTaR 在 mini-F2F 基准上将模型从低于 GPT-4 Agent 基线提升至超越它
- **关键发现**：带思考的模型从推理预算扩展（scaling inference budget）中获益更大——思考有效多样化了搜索空间
- 模型在非形式化思考中执行了形式化代码中不存在的推理步骤

此后，非形式化思考 + RL 训练已被定理证明社区广泛采用（DeepSeek-Prover 等），也是 O1、DeepSeek-R1 等推理模型的核心范式。

## 2.4 本章小结

LeanSTaR 的核心洞察：仅在形式化代码上训练可能不足以学到生成代码所需的思维过程。通过 RL 学习生成非形式化思考，可以显著提升证明能力。

## 3 第二部分：非形式化证明——草稿-草图-证明

### 3.1 动机：结合高级推理与低级证明

#### Sledgehammer：自动化证明工具

Isabelle 中的 Sledgehammer 是一种“锤子”（Hammer）工具，它将问题发送给外部自动定理证明器（一阶逻辑、高阶逻辑、SMT 求解器），非常擅长填补证明中的小缺口，但无法处理需要创造性的高级证明策略。

### 3.2 Draft, Sketch, Prove (DSP) 方法

#### 三步证明流程

1. **Draft**（草稿）：LLM 生成非形式化证明（自然语言）
2. **Sketch**（草图）：LLM 将非形式化证明翻译为形式化草图，留下待填充的缺口
3. **Prove**（证明）：低级证明器（如 Sledgehammer）填补所有缺口

若所有缺口被填补，则获得完整的形式化证明。

搜索策略不同于传统的树搜索：生成多个非形式化草稿和/或多个形式化草图翻译，尝试填补缺口，任一成功即得到证明。

### 3.3 实验结果

在 mini-F2F 上（CodeX/Minerva 时代）：

- 随着采样次数增加，性能显著提升
- 采样足够多时，LLM 生成的非形式化证明甚至超过了使用人类手写证明的表现

## 4 第三部分：为 Lean 构建 Hammer——LeanHammer

### 4.1 Hammer 管道架构

#### LeanHammer 组件

1. **前提选择**（Premise Selection）：神经检索模型嵌入目标和候选前提，用余弦相似度选择最相关的定理/定义/引理
2. **翻译层**（lean-auto）：将 Lean 的依赖类型论翻译为自动证明器的逻辑
3. **自动定理证明器**（Zipperposition 等）
4. **证明重构**（Duper）：将自动证明器的输出翻译回 Lean 证明
5. **树搜索**（AESOP）：组织整个搜索过程

用户只需在证明中调用一个 `hammer` 命令，系统自动完成前提选择、翻译、自动证明和重构。

## 4.2 实验结果

- LeanHammer 的神经前提选择器（仅 1 亿参数）显著优于先前工作
- 与符号前提选择器（Moogler）互补，取并集后性能进一步提升
- 性能正在接近使用人类证明中的真实前提时的理论上限

## 4.3 本章小结

DSP 和 LeanHammer 代表了两种互补的方法：前者通过高级草图引导证明结构，后者提供强大的低级自动化填补工具。

# 5 第四部分：迈向研究级数学的 AI 辅助

## 5.1 研究级形式化的特殊挑战

研究级数学的形式化（如 Terence Tao 的 PFR 项目）涉及：

1. 创建**蓝图**（Blueprint）：将证明分解为依赖图
2. 编写大量**新定义**和**辅助引理**
3. 组装最终定理和证明

### 当前基准测试的局限

主流基准（如 mini-F2F）基于数学竞赛题，具有自包含、依赖标准结果的特点。而真实研究项目中的定理**依赖于项目特有的新定义和新引理**，这一差距导致在竞赛题上表现优秀的模型在实际项目中可能严重失败。

## 5.2 MiniCTX 基准测试

### MiniCTX：上下文依赖的定理证明基准

- 从真实 Lean 项目（如 Mathlib、PFR）中提取测试定理
- 按日期截止选取，可持续更新以领先于模型训练截止日期
- 区分文件内依赖和跨文件依赖
- 实验表明：在竞赛题上，上下文几乎不影响性能；在真实项目上，**缺少上下文导致性能断崖式下降**

## 5.3 实用工具：LLMLean

LLMLean 是一个 Lean 插件，可连接本地模型（通过 Ollama/LM Studio）或 API 模型（OpenAI、Claude、Gemini 等），在 VS Code 中直接提供经 Lean 验证的证明建议。

## 6 总结与延伸

### 6.1 三条桥接路径

1. **非形式化思考 + 形式化证明** (LeanSTaR): 训练模型在证明步骤前进行非形式化推理
2. **非形式化证明 + 形式化填补** (DSP + LeanHammer): 先生成高级草图, 再用自动化工具填补细节
3. **研究级辅助** (MiniCTX + LLMLean): 面向真实数学项目的上下文感知证明和实用工具

### 6.2 拓展阅读

- LeanSTaR (ICLR 2025): RL 训练模型生成非形式化思考
- Draft, Sketch, Prove (ICLR 2023): 草图-填补证明方法
- MiniCTX (ICLR 2025 Oral): 上下文依赖定理证明基准
- LLMLean: <https://github.com/cmu-l3/llmlean>
- DeepSeek-Prover: 融合非形式化思考的 SOTA 证明器